

DOM — Document Object Model — HTML-এর সাথে JavaScript-এর যোগাযোগ

এখন থেকে JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ practical অংশগুলোর একটি শুরু হচ্ছে — **DOM (Document Object Model)**। এতদিন আমরা JavaScript-এর ভিতরের language concepts শিখেছি। এখন শিখব কীভাবে JavaScript দিয়ে **HTML webpage-এর content, structure, style এবং elements পরিবর্তন করা যায়।**

HTML Document → Browser DOM Tree (Nodes & Elements) → JavaScript API → Interactive Webpage

ব্রাউজারে যেকোনো আধুনিক ডায়নামিক ওয়েব অ্যাপ্লিকেশন বা রিয়াক্টি ফ্রেমওয়ার্কের মূল ভিত্তি হলো এই DOM আর্কিটেকচার। 🔥

PART 01 — DOM ARCHITECTURE, NODES & THE DOCUMENT OBJECT (15.1 – 15.10)

15.1 DOM কী? — DOM কীভাবে কাজ করে?

DOM = Document Object Model

Browser যখন একটি HTML page load করে, তখন HTML document-কে একটি **object-based tree structure** হিসেবে তৈরি করে।

ধরো HTML:

15.1 HTML

MARKUP

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
</head>

<body>

  <h1>Hello World</h1>

  <p>Welcome to JavaScript</p>

  <button>Click Me</button>

</body>
</html>
```

Browser এটাকে roughly এভাবে চিত্রা করে:

```
Document
├── html
│   ├── head
│   │   └── title
│   └── body
│       ├── h1
│       ├── p
│       └── button
```

এই structure-টাই DOM Tree।

15.2 DOM-এর Real-Life Example

ধরো একটি University campus:

```
University
├── CSE Department
│   ├── Teacher
│   ├── Student
│   └── Lab
└── EEE Department
    ├── Teacher
    └── Lab
```

এখানে:

- University = root
- Department = parent
- Teacher/Student/Lab = child

HTML DOM-ও একই ধরনের hierarchy follow করে।

15.3 JavaScript কেন DOM ব্যবহার করে?

ধরো HTML:

15.3 HTML

MARKUP

```
<h1>Hello</h1>
```

JavaScript দিয়ে তুমি এটাকে পরিবর্তন করতে পারো:

15.3 JAVASCRIPT

RUN

```
document.querySelector("h1").textContent = "Welcome Shariar!";
```

Browser-এ:

TEXT

Welcome Shariar!

অর্থাৎ:

```
HTML
↓
DOM
↓
JavaScript
↓
Change
↓
Updated Webpage
```

15.4 document কী?

DOM-এর সবচেয়ে গুরুত্বপূর্ণ object হলো:

15.4 JAVASCRIPT

RUN

```
document
```

`document` পুরো HTML webpage-কে represent করে।

Example:

15.4 JAVASCRIPT

RUN

```
console.log(document);
```

Browser console-এ পুরো HTML document দেখতে পারবে।

15.5 document.documentElement

পুরো HTML element পাওয়া যায়:

15.5 JAVASCRIPT

RUN

```
console.log(document.documentElement);
```

15.5 HTML

MARKUP

```
<html> ... </html>
```

15.6 document.head

HTML-এর `<head>` :

15.6 JAVASCRIPT

RUN

```
console.log(document.head);
```

15.6 HTML

MARKUP

```
<head> ... </head>
```

15.7 document.body

HTML-এর `<body>` :

15.7 JAVASCRIPT

RUN

```
console.log(document.body);
```

15.7 HTML

MARKUP

```
<body> ... </body>
```

15.8 DOM Element কী?

HTML-এর প্রতিটি element DOM-এর মধ্যে একটি object/node হিসেবে থাকে।

যেমন:

15.8 HTML

MARKUP

```
<h1>Hello</h1>
```

এখানে:

TEXT

```
h1 = DOM Element
```

JavaScript:

15.8 JAVASCRIPT

RUN

```
let heading = document.querySelector("h1");  
  
console.log(heading);
```

15.9 DOM Node কী?

DOM শুধু HTML element নিয়ে তৈরি নয়।

DOM-এ বিভিন্ন ধরনের node থাকতে পারে।

যেমন:

```
Document
Element
Text
Comment
Attribute
```

Example:

15.9 HTML

MARKUP

```
<h1>Hello</h1>
```

এখানে:

```
h1 → Element Node
Hello → Text Node
```

15.10 DOM Tree

ধরো:

15.10 HTML

MARKUP

```
<body>

  <h1>Hello</h1>

  <p>JavaScript</p>

</body>
```

DOM Tree:

```
document
├── html
│   └── body
│       ├── h1
│       │   └── "Hello"
│       └── p
│           └── "JavaScript"
```

PART 02 — DOM ELEMENT SELECTION & QUERY SELECTORS (15.11 – 15.24)

15.11 Element Select করা

DOM manipulation-এর প্রথম কাজ হলো:

HTML element-কে JavaScript দিয়ে খুঁজে বের করা।

সবচেয়ে গুরুত্বপূর্ণ methods:

15.11 JAVASCRIPT

RUN

```
document.getElementById()
document.getElementsByClassName()
document.getElementsByTagName()
document.querySelector()
document.querySelectorAll()
```

15.12 getElementById()

HTML:

15.12 HTML

MARKUP

```
<h1 id="title">Hello World</h1>
```

JavaScript:

15.12 JAVASCRIPT

RUN

```
let title = document.getElementById("title");  
  
console.log(title);
```

এটি `<h1>` element return করবে।

15.13 ID দিয়ে Content Change

HTML:

15.13 HTML

MARKUP

```
<h1 id="title">Hello World</h1>
```

JavaScript:

15.13 JAVASCRIPT

RUN

```
let title = document.getElementById("title");  
  
title.textContent = "Welcome to JavaScript!";
```

Browser:

TEXT

Welcome to JavaScript!

15.14 getElementsByClassName()

HTML:

15.14 HTML

MARKUP

```
<p class="text">One</p>  
<p class="text">Two</p>  
<p class="text">Three</p>
```

JavaScript:

15.14 JAVASCRIPT

RUN

```
let elements = document.getElementsByClassName("text");  
  
console.log(elements);
```

এখানে multiple elements পাওয়া যাবে।

এটি একটি **HTMLCollection** return করে।

15.15 HTMLCollection

15.15 JAVASCRIPT

RUN

```
let elements = document.getElementsByClassName("text");  
  
console.log(elements.length);
```

TEXT

3

Index দিয়ে access:

15.15 JAVASCRIPT

RUN

```
console.log(elements[0]);
```

প্রথম `<p>` পাওয়া যাবে।

15.16 getElementsByTagName()

Tag name দিয়ে element খুঁজতে:

15.16 HTML

MARKUP

```
<p>Hello</p>  
<p>JavaScript</p>  
<p>DOM</p>
```

15.16 JAVASCRIPT

RUN

```
let paragraphs = document.getElementsByTagName("p");  
  
console.log(paragraphs.length);
```

TEXT

3

15.17 querySelector()

এটি DOM-এর সবচেয়ে useful selector method-গুলোর একটি।

15.17 JAVASCRIPT

RUN

```
document.querySelector(selector);
```

ID:

15.17 JAVASCRIPT

RUN

```
let title = document.querySelector("#title");
```

Class:

15.17 JAVASCRIPT

RUN

```
let box = document.querySelector(".box");
```

Tag:

15.17 JAVASCRIPT

RUN

```
let heading = document.querySelector("h1");
```

15.18 `querySelector()` প্রথম element দেয়

HTML:

15.18 HTML

MARKUP

```
<p class="text">One</p>
<p class="text">Two</p>
<p class="text">Three</p>
```

15.18 JAVASCRIPT

RUN

```
let element = document.querySelector(".text");

console.log(element.textContent);
```

TEXT

One

শুধু প্রথম matching element পাওয়া যাবে।

15.19 `querySelectorAll()`

সব matching element পেতে:

15.19 JAVASCRIPT

RUN

```
let elements = document.querySelectorAll(".text");

console.log(elements);
```

এটি সাধারণত একটি **NodeList** return করে।

15.20 `NodeList`

15.20 JAVASCRIPT

RUN

```
let elements = document.querySelectorAll(".text");

console.log(elements.length);
```

TEXT

3

Index:

15.20 JAVASCRIPT

RUN

```
console.log(elements[1].textContent);
```

TEXT

Two

15.21 querySelector() বনাম querySelectorAll()

Method	Result
<code>querySelector()</code>	প্রথম matching element
<code>querySelectorAll()</code>	সব matching element

Example:

```
15.21 JAVASCRIPT RUN  
document.querySelector(".box");
```

→ প্রথম `.box`

```
15.21 JAVASCRIPT RUN  
document.querySelectorAll(".box");
```

→ সব `.box`

15.22 CSS Selector ব্যবহার

`querySelector()` -এ CSS selector ব্যবহার করা যায়।

HTML:

```
15.22 HTML MARKUP  
<div id="container">  
  <p class="text">Hello</p>  
</div>
```

ID

```
15.22 JAVASCRIPT RUN  
document.querySelector("#container");
```

Class

```
15.22 JAVASCRIPT RUN  
document.querySelector(".text");
```

Tag

```
15.22 JAVASCRIPT RUN  
document.querySelector("p");
```

Descendant

```
15.22 JAVASCRIPT RUN  
document.querySelector("#container p");
```

15.23 Multiple Selector

```
15.23 JAVASCRIPT RUN  
let element = document.querySelector("h1, p");
```

প্রথম matching `h1` অথবা `p` পাওয়া যাবে।

15.24 Attribute Selector

HTML:

15.24 HTML

MARKUP

```
<input type="email">
```

JavaScript:

15.24 JAVASCRIPT

RUN

```
let input = document.querySelector('input[type="email"]');
```

PART 03 — CONTENT, ATTRIBUTES & FORM ELEMENT VALUE MANIPULATION (15.25 – 15.37)

15.25 Content পরিবর্তন — `textContent`

HTML:

15.25 HTML

MARKUP

```
<h1 id="title">Old Title</h1>
```

JavaScript:

15.25 JAVASCRIPT

RUN

```
document.getElementById("title").textContent = "New Title";
```

Browser:

TEXT

New Title

15.26 innerHTML

HTML:

15.26 HTML

MARKUP

```
<div id="box"></div>
```

JavaScript:

15.26 JAVASCRIPT

RUN

```
document.getElementById("box").innerHTML = "<h2>Hello</h2>";
```

এখন HTML হবে:

15.26 HTML

MARKUP

```
<div id="box">
  <h2>Hello</h2>
</div>
```

15.27 textContent বনাম innerHTML

textContent	innerHTML
Text set করে	HTML set করতে পারে
HTML parse করে না	HTML parse করে
সাধারণত safer	User input-এর ক্ষেত্রে সাবধান
Plain text-এর জন্য ভালো	Dynamic HTML-এর জন্য useful

Example:

15.27 JAVASCRIPT

RUN

```
element.textContent = "<b>Hello</b>";
```

Browser:

TEXT

Hello

কিন্তু:

15.27 JAVASCRIPT

RUN

```
element.innerHTML = "<b>Hello</b>";
```

Browser:

Hello (bold)

15.28 innerText

15.28 JAVASCRIPT

RUN

```
element.innerText
```

এটি user-visible text-এর সাথে বেশি সম্পর্কিত।

Example:

15.28 JAVASCRIPT

RUN

```
let text = document.querySelector("p");
console.log(text.innerText);
```

★ সাধারণভাবে মনে রাখো:

```
textContent → DOM-এর text
innerText → rendered/visible text-এর দিকে বেশি focused
innerHTML → HTML structure সহ
```

15.29 Element-এর Attribute

HTML:

15.29 HTML

MARKUP

```

```

JavaScript:

15.29 JAVASCRIPT

RUN

```
let image = document.getElementById("photo");
console.log(image.src);
console.log(image.alt);
```

15.30 `getAttribute()`

15.30 JAVASCRIPT

RUN

```
let image = document.querySelector("img");  
  
console.log(image.getAttribute("src"));
```

TEXT

old.jpg

15.31 `setAttribute()`

15.31 JAVASCRIPT

RUN

```
image.setAttribute("src", "new.jpg");
```

এখন:

15.31 HTML

MARKUP

```

```

15.32 `hasAttribute()`

15.32 JAVASCRIPT

RUN

```
let image = document.querySelector("img");  
  
console.log(image.hasAttribute("alt"));
```

TEXT

true

15.33 `removeAttribute()`

15.33 JAVASCRIPT

RUN

```
image.removeAttribute("alt");
```

এখন `alt` attribute remove হয়ে যাবে।

15.34 Direct Property দিয়ে Attribute পরিবর্তন

অনেক common attribute সরাসরি property দিয়েও পরিবর্তন করা যায়।

15.34 JAVASCRIPT

RUN

```
let image = document.querySelector("img");  
  
image.src = "new.jpg";  
image.alt = "New image";
```

15.35 Input Value

HTML:

15.35 HTML

MARKUP

```
<input id="username" type="text">
```

JavaScript:

15.35 JAVASCRIPT

RUN

```
let input = document.getElementById("username");
```

```
input.value = "Shariar";
```

Input box-এ:

TEXT

Shariar

দেখাবে।

Value পড়তে:

15.35 JAVASCRIPT

RUN

```
console.log(input.value);
```

15.36 Checkbox

HTML:

15.36 HTML

MARKUP

```
<input id="agree" type="checkbox">
```

JavaScript:

15.36 JAVASCRIPT

RUN

```
let checkbox = document.getElementById("agree");
```

```
console.log(checkbox.checked);
```

TEXT

false

Check করলে:

15.36 JAVASCRIPT

RUN

```
checkbox.checked = true;
```

15.37 Select Option

HTML:

15.37 HTML

MARKUP

```
<select id="department">
```

```
  <option value="cse">CSE</option>
```

```
  <option value="eee">EEE</option>
```

```
</select>
```

JavaScript:

15.37 JAVASCRIPT

RUN

```
let department = document.getElementById("department");
```

```
console.log(department.value);
```

15.38 CSS পরিবর্তন — `style`

HTML:

15.38 HTML

MARKUP

```
<h1 id="title">Hello</h1>
```

JavaScript:

15.38 JAVASCRIPT

RUN

```
let title = document.getElementById("title");

title.style.color = "red";
title.style.fontSize = "40px";
```

এখন heading লাল এবং বড় হবে।

15.39 CSS Property Naming

CSS:

15.39 CSS

STYLESHEET

```
background-color: blue;
```

JavaScript:

15.39 JAVASCRIPT

RUN

```
element.style.backgroundColor = "blue";
```

CSS:

15.39 CSS

STYLESHEET

```
font-size: 30px;
```

JavaScript:

15.39 JAVASCRIPT

RUN

```
element.style.fontSize = "30px";
```

CSS:

15.39 CSS

STYLESHEET

```
margin-top: 20px;
```

JavaScript:

15.39 JAVASCRIPT

RUN

```
element.style.marginTop = "20px";
```

অর্থাৎ CSS-এর `kebab-case` → JavaScript-এ `camelCase` ।

15.40 classList

Direct style পরিবর্তনের চেয়ে অনেক সময় CSS class ব্যবহার করা ভালো।

HTML:

15.40 HTML

MARKUP

```
<div id="box">Hello</div>
```

CSS:

15.40 CSS

STYLESHEET

```
.active {  
  background-color: blue;  
  color: white;  
}
```

JavaScript:

15.40 JAVASCRIPT

RUN

```
let box = document.getElementById("box");  
  
box.classList.add("active");
```

15.41 classList.add()

15.41 JAVASCRIPT

RUN

```
element.classList.add("active");
```

একটি class যোগ করে।

15.42 classList.remove()

15.42 JAVASCRIPT

RUN

```
element.classList.remove("active");
```

Class remove করে।

15.43 classList.toggle()

15.43 JAVASCRIPT

RUN

```
element.classList.toggle("active");
```

যদি class থাকে:

TEXT

remove

না থাকলে:

TEXT

add

এটি খুব usefull

15.44 classList.contains()

15.44 JAVASCRIPT

RUN

```
console.log(element.classList.contains("active"));
```

TEXT

true

অথবা:

TEXT

false

15.45 classList.replace()

15.45 JAVASCRIPT

RUN

```
element.classList.replace("old", "new");
```

একটি class অন্য class দিয়ে replace করবে।

PART 05 — ELEMENT CREATION, INSERTION, REMOVAL & CLONING (15.46 – 15.57)

15.46 Element তৈরি করা

JavaScript দিয়ে নতুন HTML element তৈরি করা যায়।

15.46 JAVASCRIPT

RUN

```
let paragraph = document.createElement("p");
```

এখন `paragraph` একটি নতুন DOM element।

15.47 নতুন Element-এর Content

15.47 JAVASCRIPT

RUN

```
let paragraph = document.createElement("p");  
paragraph.textContent = "Hello JavaScript";
```

কিন্তু এখনও webpage-এ দেখা যাবে না।

কারণ element-টাকে DOM-এ add করতে হবে।

15.48 append()

HTML:

15.48 HTML

MARKUP

```
<div id="container"></div>
```

JavaScript:

15.48 JAVASCRIPT

RUN

```
let container = document.getElementById("container");

let paragraph = document.createElement("p");

paragraph.textContent = "Hello JavaScript";

container.append(paragraph);
```

DOM:

15.48 HTML

MARKUP

```
<div id="container">
  <p>Hello JavaScript</p>
</div>
```

15.49 appendChild()

15.49 JAVASCRIPT

RUN

```
container.appendChild(paragraph);
```

এটিও child element add করে।

15.50 append() বনাম appendChild()

append()	appendChild()
Node এবং text দুটোই নিতে পারে	Node নেয়
একাধিক item নিতে পারে	সাধারণত একটি node
Modern ও flexible	Traditional DOM API

Example:

15.50 JAVASCRIPT

RUN

```
container.append("Hello");
```

এটা সম্ভব।

কিন্তু:

15.50 JAVASCRIPT

RUN

```
container.appendChild("Hello");
```

এটা valid node না হওয়ায় কাজ করবে না।

15.51 prepend()

Parent-এর শুরুতে element যোগ করতে:

15.51 JAVASCRIPT

RUN

```
container.prepend(paragraph);
```

15.52 before()

কোনো element-এর আগে কিছু যোগ করতে:

```
15.52 JAVASCRIPT
```

RUN

```
element.before(paragraph);
```

15.53 after()

কোনো element-এর পরে:

```
15.53 JAVASCRIPT
```

RUN

```
element.after(paragraph);
```

15.54 Element Remove

Modern method:

```
15.54 JAVASCRIPT
```

RUN

```
element.remove();
```

Example:

```
15.54 JAVASCRIPT
```

RUN

```
let paragraph = document.querySelector("p");  
  
paragraph.remove();
```

15.55 removeChild()

Parent থেকে child remove:

```
15.55 JAVASCRIPT
```

RUN

```
container.removeChild(paragraph);
```

15.56 Element Replace

```
15.56 JAVASCRIPT
```

RUN

```
container.replaceChildren(paragraph);
```

এটি container-এর existing children replace করে।

আর একটি নির্দিষ্ট child replace করতে:

```
15.56 JAVASCRIPT
```

RUN

```
oldElement.replaceWith(newElement);
```

15.57 Clone করা

একটি DOM element copy করতে:

```
15.57 JAVASCRIPT
```

RUN

```
let copy = element.cloneNode(true);
```

এখানে:

```
true → children-সহ clone  
false → শুধু element clone
```

15.58 Parent খুঁজে বের করা

HTML:

15.58 HTML

MARKUP

```
<div id="parent">
  <p id="child">Hello</p>
</div>
```

JavaScript:

15.58 JAVASCRIPT

RUN

```
let child = document.getElementById("child");
console.log(child.parentElement);
```

div পাওয়া যাবে।

15.59 Children

15.59 JAVASCRIPT

RUN

```
let parent = document.getElementById("parent");
console.log(parent.children);
```

এটি শুধু element children দেখায়।

15.60 firstElementChild

15.60 JAVASCRIPT

RUN

```
console.log(parent.firstElementChild);
```

15.61 lastElementChild

15.61 JAVASCRIPT

RUN

```
console.log(parent.lastElementChild);
```

15.62 nextElementSibling

HTML:

15.62 HTML

MARKUP

```
<p>One</p>
<p id="current">Two</p>
<p>Three</p>
```

JavaScript:

15.62 JAVASCRIPT

RUN

```
let current = document.getElementById("current");
console.log(current.nextElementSibling);
```

15.62 HTML

MARKUP

```
<p>Three</p>
```

15.63 previousElementSibling

15.63 JAVASCRIPT

RUN

```
console.log(current.previousElementSibling);
```

15.63 HTML

MARKUP

```
<p>One</p>
```

15.64 DOM Traversal

DOM traversal মানে DOM Tree-এর মধ্যে এক element থেকে অন্য element-এ যাওয়া।

Important properties:

15.64 JAVASCRIPT

RUN

```
parentElement  
children  
firstElementChild  
lastElementChild  
nextElementSibling  
previousElementSibling
```

15.65 children বনাম childNodes

15.65 JAVASCRIPT

RUN

```
element.children
```

→ শুধু element children।

15.65 JAVASCRIPT

RUN

```
element.childNodes
```

→ text node, comment node সহ বিভিন্ন node থাকতে পারে।

Example:

15.65 HTML

MARKUP

```
<div>  
  Hello  
  <p>World</p>  
</div>
```

এখানে whitespace/text node থাকার কারণে `childNodes` এবং `children` -এর result একই নাও হতে পারে।

15.66 closest()

কোনো element-এর কাছে matching ancestor খুঁজতে:

15.66 JAVASCRIPT

RUN

```
let button = document.querySelector("button");  
  
let card = button.closest(".card");
```

Real-world application-এ এটি খুব useful।

15.67 matches()

কোনো element CSS selector match করছে কিনা:

15.67 JAVASCRIPT

RUN

```
if (element.matches(".active")) {  
  console.log("Active");  
}
```

15.68 DOM Mini Project — Counter

এখন একটি ছোট project করি।

HTML

15.68 HTML

MARKUP

```
<h1 id="count">0</h1>

<button id="increase">+</button>
<button id="decrease">-</button>
```

JavaScript

15.68 JAVASCRIPT

RUN

```
let count = 0;

let countElement = document.getElementById("count");

let increaseButton =
  document.getElementById("increase");

let decreaseButton =
  document.getElementById("decrease");

increaseButton.addEventListener("click", function () {

  count++;

  countElement.textContent = count;

});

decreaseButton.addEventListener("click", function () {

  count--;

  countElement.textContent = count;

});
```

এখানে আমরা `addEventListener()` ব্যবহার করেছি, যেটা Chapter 16-এ বিস্তারিত শিখবে।

15.69 DOM Mini Project — Dynamic List

HTML:

15.69 HTML

MARKUP

```
<input id="taskInput" type="text">

<button id="addButton">Add Task</button>

<ul id="taskList"></ul>
```

JavaScript:

15.69 JAVASCRIPT

RUN

```
let input = document.getElementById("taskInput");
let button = document.getElementById("addButton");
let list = document.getElementById("taskList");

button.addEventListener("click", function () {

    let task = input.value;

    if (task === "") {
        return;
    }

    let li = document.createElement("li");

    li.textContent = task;

    list.append(li);

    input.value = "";

});
```

User লিখবে:

TEXT

Learn JavaScript

তারপর Add চাপলে:

TEXT

• Learn JavaScript

এভাবে dynamic list তৈরি হবে।

15.70 DOM Mini Project — Dark Mode

HTML:

15.70 HTML

MARKUP

```
<button id="themeButton">
  Toggle Dark Mode
</button>
```

CSS:

15.70 CSS

STYLESHEET

```
.dark {
  background-color: black;
  color: white;
}
```

JavaScript:

15.70 JAVASCRIPT

RUN

```
let button = document.getElementById("themeButton");

button.addEventListener("click", function () {

  document.body.classList.toggle("dark");

});
```

Button click করলে:

```
Light → Dark
Dark → Light
```

15.71 dataset

HTML:

15.71 HTML

MARKUP

```
<button
  id="product"
  data-id="101"
  data-category="laptop">
  Laptop
</button>
```

JavaScript:

15.71 JAVASCRIPT

RUN

```
let product = document.getElementById("product");

console.log(product.dataset.id);
console.log(product.dataset.category);
```

TEXT

```
101
laptop
```

`data-*` attributes custom data রাখার জন্য usefull

15.72 DOM-এর কিছু গুরুত্বপূর্ণ Properties

Property	কাজ
<code>textContent</code>	Text পড়া/লেখা
<code>innerHTML</code>	HTML পড়া/লেখা
<code>innerText</code>	Visible text
<code>value</code>	Input value
<code>style</code>	Inline CSS
<code>classList</code>	CSS class manage
<code>id</code>	Element ID
<code>children</code>	Element children
<code>parentElement</code>	Parent
<code>dataset</code>	<code>data-*</code> values

15.73 DOM-এর গুরুত্বপূর্ণ Methods

Method	কাজ
<code>getElementById()</code>	ID দিয়ে select
<code>getElementsByClassName()</code>	Class দিয়ে select
<code>getElementsByTagName()</code>	Tag দিয়ে select
<code>querySelector()</code>	প্রথম matching element
<code>querySelectorAll()</code>	সব matching elements
<code>createElement()</code>	Element তৈরি
<code>append()</code>	শেষে add
<code>prepend()</code>	শুরুতে add
<code>remove()</code>	Remove
<code>replaceWith()</code>	Replace
<code>cloneNode()</code>	Clone
<code>closest()</code>	Ancestor খোঁজা
<code>matches()</code>	Selector match check
<code>getAttribute()</code>	Attribute পড়া
<code>setAttribute()</code>	Attribute set
<code>removeAttribute()</code>	Attribute remove
<code>hasAttribute()</code>	Attribute আছে কিনা

15.74 innerHTML ব্যবহারে সতর্কতা

ধরো user input:

15.74 JAVASCRIPT

RUN

```
let userInput = "<img src=x onerror=alert('Hacked')>";
```

এটা সরাসরি:

15.74 JAVASCRIPT

RUN

```
element.innerHTML = userInput;
```

দিলে security problem হতে পারে।

এ ধরনের issue-কে সাধারণভাবে **XSS (Cross-Site Scripting)** বলা হয়।

User-provided plain text দেখানোর ক্ষেত্রে:

15.74 JAVASCRIPT

RUN

```
element.textContent = userInput;
```

সাধারণত safer approach।

Security chapter-এ আমরা XSS আরও বিস্তারিত দেখব।

15.75 Static vs Live Collections

এটি একটু advanced কিন্তু গুরুত্বপূর্ণ।

`getElementsByClassName()` সাধারণত **live HTMLCollection** দেয়।

15.75 JAVASCRIPT

RUN

```
let items = document.getElementsByClassName("item");
```

DOM পরিবর্তন হলে collection-এর content automatically update হতে পারে।

অন্যদিকে:

15.75 JAVASCRIPT

RUN

```
let items = document.querySelectorAll(".item");
```

সাধারণত **static NodeList** দেয়।

অর্থাৎ পরে DOM-এ নতুন matching element যোগ করলেও আগের NodeList automatically update হবে না।

15.76 DOM Ready হওয়ার বিষয়

যদি JavaScript HTML element তৈরি হওয়ার আগেই execute করে:

15.76 JAVASCRIPT

RUN

```
let title = document.getElementById("title");

console.log(title);
```

তাহলে `null` পাওয়া যেতে পারে।

একটি common solution হলো script-কে HTML-এর শেষে রাখা:

15.76 HTML

MARKUP

```
<body>

  <h1 id="title">Hello</h1>

  <script src="script.js"></script>

</body>
```

আরেকটি approach হলো:

15.76 JAVASCRIPT

RUN

```
document.addEventListener("DOMContentLoaded", function () {

  let title = document.getElementById("title");

  console.log(title);

});
```

`DOMContentLoaded` আমরা Events chapter-এ আরও বিস্তারিত দেখব।

15.77 DOM-এর পুরো Mental Model

একটি webpage-কে এভাবে চিন্তা করো:

```
HTML
↓
Browser parses HTML
↓
DOM Tree তৈরি
↓
JavaScript DOM select করে
↓
Content / Attribute / Style / Structure change
↓
Browser updated UI দেখায়
```

15.78 Must Know

JavaScript দিয়ে frontend development করতে হলে এগুলো অবশ্যই ভালোভাবে জানতে হবে:

Selection

15.78 JAVASCRIPT

RUN

```
getElementById()  
getElementsByClassName()  
getElementsByName()  
querySelector()  
querySelectorAll()
```

Content

15.78 JAVASCRIPT

RUN

```
textContent  
innerText  
innerHTML
```

Attributes

15.78 JAVASCRIPT

RUN

```
getAttribute()  
setAttribute()  
hasAttribute()  
removeAttribute()
```

Classes

15.78 JAVASCRIPT

RUN

```
classList.add()  
classList.remove()  
classList.toggle()  
classList.contains()  
classList.replace()
```

Style

15.78 JAVASCRIPT

RUN

```
element.style.color  
element.style.backgroundColor  
element.style.fontSize
```

Create

15.78 JAVASCRIPT

RUN

```
document.createElement()
```

Insert

15.78 JAVASCRIPT

RUN

```
append()  
appendChild()  
prepend()  
before()  
after()
```

Remove

15.78 JAVASCRIPT

RUN

```
remove()  
removeChild()
```

Traversal

15.78 JAVASCRIPT

RUN

```
parentElement  
children  
firstElementChild  
lastElementChild  
nextElementSibling  
previousElementSibling  
closest()
```

Data

15.78 JAVASCRIPT

RUN

dataset

PART 09 — QUICK CHEAT SHEET, PRACTICE LAB (12 PROBLEMS) & FINAL SUMMARY

CHEAT SHEET

🧠 Quick Cheat Sheet — Core DOM Syntax

JAVASCRIPT

RUN

```
// Select
document.getElementById("title");

document.querySelector(".box");

document.querySelectorAll(".item");

// Content
element.textContent = "Hello";

element.innerHTML = "<b>Hello</b>";

// Attribute
element.setAttribute("title", "Hello");

element.getAttribute("title");

element.removeAttribute("title");

// Class
element.classList.add("active");

element.classList.remove("active");

element.classList.toggle("active");

// Style
element.style.color = "red";

// Create
let div = document.createElement("div");

// Add
parent.append(div);

// Remove
div.remove();

// Parent
element.parentElement;

// Children
element.children;

// Data
element.dataset.id;
```

LEVEL: BEGINNER

1. একটি `<h1>` element তৈরি করে JavaScript দিয়ে তার text পরিবর্তন করো।
2. একটি paragraph-এর color JavaScript দিয়ে পরিবর্তন করো।
3. একটি button-এর text পরিবর্তন করো।
4. একটি image-এর `src` এবং `alt` JavaScript দিয়ে পরিবর্তন করো।

LEVEL: INTERMEDIATE

5. একটি input থেকে name নিয়ে `<h2>` -এ দেখাও।
6. JavaScript দিয়ে নতুন `<li>` তৈরি করে `<ul>` -এ add করো।
7. একটি button click করলে একটি CSS class toggle করো।
8. পাঁচটি paragraph select করে প্রত্যেকটির text পরিবর্তন করো।

LEVEL: ADVANCED

9. একটি dynamic Todo List বানাও:

```
Input
↓
Add Task
↓
Task List
↓
Delete Task
```

10. একটি Product Card JavaScript দিয়ে dynamically তৈরি করো:

```
Product Image
Product Name
Price
Add to Cart Button
```

11. `data-id` ব্যবহার করে প্রতিটি product-এর ID store করো।
12. `closest()` ব্যবহার করে একটি card-এর parent element বের করো।

SUMMARY

Chapter 15 Final Summary — DOM Architecture & Lifecycle

আজ পর্যন্ত আমরা JavaScript-এর ভিতরের language concepts থেকে এসে **Browser-এর HTML-এর সাথে JavaScript-এর connection** শিখলাম।

সবচেয়ে গুরুত্বপূর্ণ flow:

```
document
  ↓
Select Element
  ↓
Read / Modify
  ↓
Content
Attribute
Style
Class
  ↓
Create / Add / Remove
  ↓
Dynamic Webpage
```

আর DOM-এর সবচেয়ে গুরুত্বপূর্ণ ৫টি skill:

TEXT

1. Select
2. Read
3. Modify
4. Create
5. Remove

একজন **Frontend / Full Stack Developer** হিসেবে DOM খুব ভালোভাবে জানা অত্যন্ত গুরুত্বপূর্ণ, কারণ React শেখার আগেও বুঝতে হবে browser-এর HTML/UI আসলে কীভাবে কাজ করে।

NEXT CHAPTER

Chapter 16 Preview — JavaScript Events & Event Handling

পরবর্তী: Chapter 16 — JavaScript Events & Event Handling এখানে আমরা `click`, `input`, `change`, `submit`, keyboard events, mouse events, event object, `addEventListener()`, event bubbling, capturing, delegation, `preventDefault()`, `stopPropagation()` —সব বিস্তারিত শিখব।